

FIT A BIGGER MODEL ON THE GPU YOU ALREADY HAVE

LLM Quantization Cheat Sheet

Formats, methods, naming schemes and the quality you actually lose. Everything you need to pick a number of bits and defend it.

⚙️ FP16 to INT4

🔗 GPTQ / AWQ / GGUF

📖 VRAM math



AI Coach John

FOLLOW



🔄 REPOST

Fewer Bits Per Number

DEFINITION

A model is a pile of numbers. Quantization stores each number in less space and accepts a small rounding error in exchange.

THE SAME WEIGHT, STORED FOUR WAYS

FP32		0.4271934
FP16		0.42718
INT8		0.4275
INT4	 same meaning, coarser resolution	0.437

4x

smaller from FP32 to INT8

8x

smaller from FP32 to INT4

4x

less memory traffic per token

STORED PER GROUP

WHAT IT DOES

scale

maps the integer grid back to real values

zero point

shifts the grid when weights are not centred

group size

how many weights share one scale, often 128

i Quantization does not remove parameters. The model keeps every weight it had. Each weight is simply described with less precision.



AI Coach John

FOLLOW



REPOST

The Only Formula You Need

VRAM MATH

Parameters times bytes per parameter. Everything else in capacity planning is a correction on top of this line.

```
# weights only
vram_gb = params_b * bytes_per_param

# a 7B model, four ways
FP16  7 * 2.0 = 14.0 GB    # needs a 24GB card
INT8   7 * 1.0 = 7.0 GB    # fits a 12GB card
INT4   7 * 0.5 = 3.5 GB    # fits an 8GB card

# then add roughly 20 to 40 percent for KV cache,
# activations and framework overhead
```

FORMAT	BYTES / PARAM	70B MODEL
FP32	4.0	280 GB
FP16 / BF16	2.0	140 GB
FP8 / INT8	1.0	70 GB
INT4	0.5	35 GB

⚠ Real quantized files are slightly larger than the formula suggests. Scales and zero points are stored per group, and some layers stay in higher precision.



The Number Formats

REFERENCE

Two things vary: how many bits, and how they split between range and precision.

FORMAT	CHARACTER	USED FOR
FP32	full precision, the reference point	rarely, too wasteful
FP16	half the bits, narrow range, can overflow	inference
BF16	same range as FP32, less precision, stable	training default
FP8	hardware accelerated on recent GPUs	fast serving
INT8	integers plus a scale factor	the safe workhorse
INT4	sixteen possible values per weight	fitting big models
NF4	4 bit spacing tuned to a normal distribution	QLoRA training



BF16 trades precision for range



FP16 trades range for precision



Integer formats are never raw integers. Each group carries a scale, and often a zero point, mapping them back to real values.



Weights Are Not The Whole Bill

VRAM BUDGET

Quantizing weights fixes one line of a three line budget. At long context it is not even the biggest line.

SHORT CONTEXT, SMALL BATCH



LONG CONTEXT, LARGE BATCH



- Weights, fixed
- KV cache, grows with context and batch
- Activations and framework

```
# the line people forget to compute
kv_gb = 2 * layers * kv_heads * head_dim * seq_len * batch * bytes / 1e9
#      ^ keys and values                      ^ 2 for FP16
# it scales linearly with context AND with batch size
```



Weights are fixed once loaded



The cache grows all session long



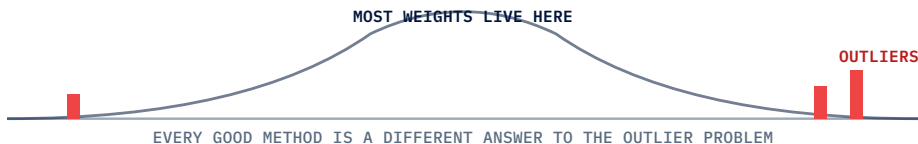
The classic trap: you quantize to INT4, celebrate the free VRAM, then fill all of it with KV cache and hit out of memory at 32k context anyway.



Why You Can Get Away With It INTUITION

Neural networks are redundant and noise tolerant. Most weights genuinely do not need seven decimal places.

- 01 **Weights cluster near zero.** Most values sit in a narrow band, so a coarse grid covers them well.
- 02 **The network is redundant.** Many weights encode overlapping information, so small errors cancel rather than compound.
- 03 **Grouping keeps errors local.** A scale per 64 or 128 weights means one large value does not ruin the whole tensor.
- 04 **But outliers matter enormously.** A tiny fraction of channels carry large activations, and crushing them wrecks quality.



This is the whole research field in one line. GPTQ, AWQ and SmoothQuant differ mainly in how they protect the small number of weights that actually matter.



Two Ways To Get There

PTQ VS QAT

Quantize a finished model, or teach the model to expect quantization while it trains. Almost everyone does the first.

POST TRAINING (PTQ)

- Take released weights and convert
- Minutes to hours on one GPU
- Needs a small calibration set, or none
- Slight quality loss at 4 bit
- What every download on Hugging Face is

QUANTIZATION AWARE (QAT)

- Simulates low precision during training
- Full training cost, needs the data
- Best quality at very low bit widths
- Only the model producer can do it
- Rare, and worth it below 4 bit

✓ Start with post training quantization, always. It is nearly free and usually enough.

🔧 Calibration matters more than people expect. A few hundred samples that look like your real traffic beat a large generic corpus.



The Method Zoo, Sorted

LANDSCAPE

Six names cover almost everything you will meet.

METHOD	IDEA	BEST FOR
GPTQ	layer by layer, minimise output error	GPU, 4 bit weights
AWQ	protect the channels activations care about	GPU, strong 4 bit
GGUF	a file format plus a family of k-quants	CPU and laptops
bitsandbytes	load in 4 bit on the fly, NF4	QLoRA training
SmoothQuant	shift outliers from activations into weights	INT8 weight and activation
FP8	native hardware format, minimal loss	modern datacentre GPUs



GPU serving, use AWQ or FP8



Local machine, use GGUF



Check engine support before picking a method. A quantization your runtime cannot load is worth nothing.



AI Coach John

FOLLOW

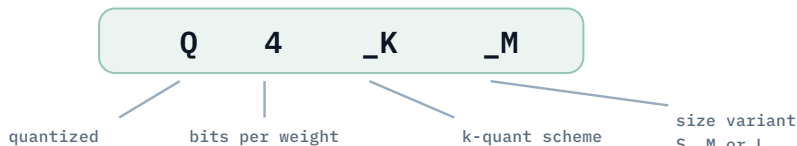


REPOST

Decoding Q4_K_M

GGUF

The most confusing filenames in local AI, explained once. Every part of the name means something.



NAME	WHAT YOU GET	VERDICT
Q8_0	8 bit, almost indistinguishable from FP16	safe
Q6_K	6 bit, very close to the original	safe
Q5_K_M	good balance when you have the room	good
Q4_K_M	the community default, best size to quality	start here
Q4_K_S	smaller, slightly worse than K_M	ok
Q3_K_M	noticeable degradation begins	careful
Q2_K	runs, but reasoning falls apart	avoid

★ If you do not want to think about it, take Q4_K_M. It is the default for a reason, and only move up or down once an eval tells you to.



What Exactly Gets Quantized

W AND A

You will see labels like W4A16 and W8A8. The first number is the weights, the second is the activations flowing through them.

LABEL	MEANING	WINS AT
W16A16	no quantization, the baseline	reference quality
W8A16	8 bit weights, full precision math	safe memory saving
W4A16	4 bit weights, dequantized to compute	single stream decode
W8A8	both quantized, integer tensor cores	large batch throughput
FP8 e4m3	native 8 bit floats end to end	modern GPU serving

WEIGHT ONLY HELPS

- Decode, one user at a time
- Bottleneck is reading weights
- Fewer bytes means more tokens per second

WEIGHT ONLY DOES NOT HELP

- Prefill of a very long prompt
- Large batch serving
- There you need faster math, not fewer bytes

i Weight only quantization has to convert back to FP16 to multiply. You save memory traffic, not arithmetic.



What Actually Degrades HONEST

Quality loss is not spread evenly. Simple tasks survive aggressive quantization, and hard tasks break first.

DAMAGE AT 4 BIT, BY TASK TYPE

classify		tiny
summarize		small
rag answer		some
code		real
math		real
long agent		worst

01 **Errors compound over length.** One slightly wrong token shifts everything after it, so long outputs suffer most.

02 **Small models suffer more.** A 3B model has less redundancy to spare than a 70B model.

03 **Exactness breaks first.** Arithmetic, syntax and structured output degrade before prose ever sounds wrong.

⚠ Quantized models fail quietly. The prose still reads fluently while the JSON key is subtly wrong, which is exactly the failure mode humans miss in review.



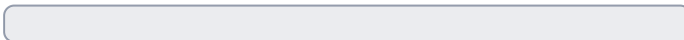
Does It Actually Run Faster

PERFORMANCE

Usually yes, and for a reason that surprises people. The speed comes from moving fewer bytes, not from doing less maths.

BYTES READ TO GENERATE ONE TOKEN, 7B MODEL

FP16



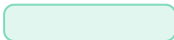
14 GB

INT8



7 GB

INT4



3.5 GB

LESS TRAFFIC, MORE TOKENS PER SECOND

SITUATION

EFFECT OF 4 BIT WEIGHTS

Single user, decoding

clearly faster, memory bound

Long prompt prefill

little change, compute bound

Very large batch

little change, or slower

Model no longer fits without it

infinitely faster than not running

Unsupported kernel path

can be slower than FP16



Aggressive quantization on a runtime without an optimised kernel for that format is a real and common way to make your model slower. Benchmark, do not assume.



AI Coach John

FOLLOW



REPOST

Quantize The Cache Too

KV QUANT

A separate setting that most people never touch, and the one that actually buys you long context.

FP16

default cache precision

FP8

half the cache, minimal loss

INT4

quarter the cache, measurable loss

```
# vLLM, quantize weights and cache independently
llm = LLM(
    model="meta-llama/Llama-3.1-8B-Instruct",
    quantization="awq",           # the weights
    kv_cache_dtype="fp8",        # the cache
    max_model_len=32768)
```



Roughly doubles the context you can hold



Or roughly doubles your batch size



If your problem is "it dies at long context" rather than "it does not load at all", cache quantization is the lever you want, not smaller weights.



How Many Bits Do You Need

DECISION

Work down from the top until it fits. Stop at the first row that runs on your hardware.

16 **FP16 or BF16.** Use it if the model fits with room to spare. There is no reason to lose quality you are not forced to lose.

8 **FP8 or INT8.** Quality loss is very hard to measure. This is the default for serious serving, and the safest step down.

4 **INT4, AWQ or Q4_K_M.** The sweet spot for local and cost constrained use. Small but real loss on hard tasks.

3 **3 bit.** Only when nothing else fits, and only for forgiving tasks. Test it properly before you trust it.

2 **2 bit.** Interesting research, not a production choice. Prefer a smaller model at 4 bit instead.



The most useful rule in the whole field: **a bigger model at 4 bit usually beats a smaller model at 16 bit**, at the same memory budget. Trade precision before you trade parameters.



What Fits On What

PLANNING

Weights only. Add KV cache headroom before committing to a card.

VRAM	FP16	INT8	INT4
8 GB	3B	7B	13B
12 GB	3B	8B	14B
24 GB	8B	14B	32B
48 GB	14B	32B	70B
80 GB	32B	70B	120B
2 x 80 GB	70B	120B	200B plus

ALSO BUDGET FOR	TYPICAL SIZE
KV cache at 32k context	several GB, model dependent
CUDA context and framework	roughly 1 to 2 GB



Leave 20 to 30 percent for the cache



Fits at 4k does not mean fits at 128k



Treat this as a first guess for shortlisting hardware. Real numbers depend on the architecture, the attention scheme and your serving engine.



AI Coach John

FOLLOW



REPOST

Proving It Still Works

EVALUATION

Never trust a quantization because the download page says the loss is negligible. Measure it on the work you actually do.

- 01 **Perplexity is a weak signal.** It barely moves while task accuracy falls. Useful as a smoke test, useless as a decision.
- 02 **Build a task eval from real traffic.** Fifty to two hundred examples with known good answers is enough to see the difference.
- 03 **Test the format, not just the content.** Check JSON validity, schema conformance and tool call correctness as separate rates.
- 04 **Test at your real context length.** Damage grows with sequence length, so a short prompt test hides the problem.
- 05 **Compare against the same model at FP16.** Not against a different model, and not against your memory of last month.

✓ Under 1 percent drop, ship it

✗ Over 5 percent drop, go up a level

🔧 An afternoon building this harness pays for itself the first time you need to change quantization, model or serving engine without guessing.



Three Commands That Cover Most Cases

PRACTICAL

Serving on a GPU, running on a laptop, or fine tuning on one card. Pick your row.

```
# 1. serve a 4 bit model on a GPU
vllm serve TheModel-AWQ --quantization awq \
  --kv-cache-dtype fp8 --max-model-len 32768

# 2. run locally on CPU or Apple silicon
llama-cli -m model-Q4_K_M.gguf -c 8192 -ngl 99

# 3. load 4 bit for QLoRA fine tuning
from transformers import BitsAndBytesConfig
cfg = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_compute_dtype=torch.bfloat16,
    bnb_4bit_use_double_quant=True)
```



Prefer a published quant over rolling your own



-ngl controls layers offloaded to GPU



Double quantization in the last recipe quantizes the scale factors as well. It saves a further half gigabyte or so on a 7B model, for free.



Seven Ways People Get This Wrong

PITFALLS

Every one of these has cost a team a week somewhere.

MISTAKE	WHAT TO DO INSTEAD
Quantizing when the model already fits	keep the precision you paid for
Sizing VRAM from weights alone	add KV cache and overhead
Judging quality by vibes on five prompts	run a real eval set
Going straight to 4 bit	try 8 bit first, it is nearly free
Ignoring what the runtime supports	match method to engine kernels
Assuming smaller is always faster	benchmark, kernels vary
Quantizing a model already fine tuned badly	fix the fine tune first



The order that works: **make it correct at full precision, then quantize, then measure, then ship.** Reversing any two of those steps is how weeks disappear.



Four Questions, One Answer

DECIDE

Run down these in order and you will land on the right configuration almost every time.

Q1 Does it fit at BF16 with cache headroom? Yes, stop. Do not quantize a model that already fits comfortably.

Q2 Does it fit at 8 bit? Yes, take FP8 on a modern GPU or INT8 otherwise. This is where most serving should live.

Q3 Do you need 4 bit to fit? Take AWQ on a GPU or Q4_K_M locally, then run your eval before anything ships.

Q4 Still out of memory? Quantize the KV cache and shorten max context before you drop to 3 bit weights.

! If none of that fits, the honest answer is a smaller model or a bigger GPU

 GPU serving, prefer FP8 or AWQ

 Local machine, prefer GGUF

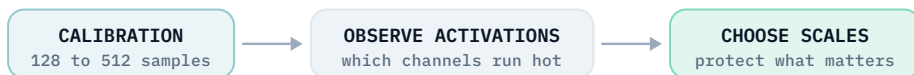
 Notice that cache quantization sits ahead of very low bit weights. It costs far less quality per gigabyte recovered.



The Data Nobody Talks About

CALIBRATION SET

Most 4 bit methods need a few hundred sample texts to decide how to round. That small choice moves quality more than people expect.



THE MODEL NEVER TRAINS. IT ONLY WATCHES ITSELF RUN.
which is why post training quantization takes minutes, not days

01

Use text that looks like your traffic. Calibrating a code assistant on web prose leaves accuracy on the table.

02

A few hundred samples is enough. More rarely helps, and the run gets slower for nothing.

03

Match the sequence length. Calibrate near the context length you actually serve at.

🎯 This is the cheapest quality win in the whole deck. Swapping a generic calibration set for a few hundred of your own prompts costs one afternoon.



Three Answers To The Outlier Problem

HOW METHODS DIFFER

A small fraction of channels carry unusually large values. Every serious method is a different way of not crushing them.

METHOD	ITS ANSWER TO OUTLIERS
GPTQ	round greedily, then correct the remaining weights to cancel the error
AWQ	find the channels activations care about and scale them up before rounding
SmoothQuant	move the difficulty out of activations and into weights, which tolerate it
Mixed precision	simply keep the worst offending layers at higher precision

BEFORE: ACTIVATIONS ARE SPIKY, WEIGHTS ARE FLAT

 activations, hard to quantize
 weights, easy

AFTER: THE DIFFICULTY IS SHARED, AND BOTH SIDES QUANTIZE WELL

i You rarely choose these by theory. Pick the one your serving engine has a fast kernel for, then verify on your own eval set.

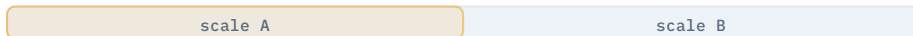


Group Size, The Hidden Setting

GRANULARITY

How many weights share one scale factor. Smaller groups mean better accuracy and a slightly larger file, and it is the setting people never touch.

GROUP SIZE 256, ONE SCALE FOR A WIDE RANGE OF VALUES



GROUP SIZE 64, FOUR TIMES THE SCALES, MUCH TIGHTER FIT



GROUP SIZE	EFFECT	VERDICT
32	best accuracy, largest overhead	niche
64	a good trade when quality matters	safe
128	the common default everywhere	start here
per channel	smallest file, noticeably worse at 4 bit	careful



Smaller groups add a little file size



And buy real accuracy at 4 bit



If a 4 bit model is close but not quite good enough, dropping group size from 128 to 64 is worth trying before you jump to 8 bit.



AI Coach John

FOLLOW



REPOST

Not Every Layer Gets Quantized

MIXED PRECISION

A "4 bit model" is not 4 bit throughout. Several parts are deliberately left alone because quantizing them costs far more than it saves.

COMPONENT	USUALLY	WHY
Attention and MLP weights	4 bit	the bulk of the parameters
Embedding table	8 bit or 16	sensitive, and shared
Output head	8 bit or 16	directly shapes every token
Layer norms	16 bit	tiny, and numerically fragile
First and last blocks	often higher	errors here propagate furthest



This is why files are larger than the formula



And why 4 bit works better than it should



Norms and the output head are a rounding error in size and a large share of the damage if quantized badly. Leaving them alone is nearly free.



Benchmarking It Honestly

PROTOCOL

Most quantization speed claims are measured wrong. Six rules make your numbers mean something.

- 01 **Warm up first.** The first few requests include kernel compilation and weight loading. Discard them.
- 02 **Separate prefill from decode.** Report time to first token and tokens per second as two numbers, never one.
- 03 **Fix the output length.** Otherwise you are comparing answer verbosity, not model speed.
- 04 **Test at your real batch size.** A single stream and a batch of 32 rank the formats differently.
- 05 **Test at your real context length.** Speed at 512 tokens tells you nothing about 32k.
- 06 **Report p95, not the mean.** The tail is what your users actually experience.

🔧 If a quantized model benchmarks slower than FP16, you have almost certainly hit a missing fast kernel for that format. Check engine support before blaming the method.



Quantizing To Train Is Not The Same

DO NOT MIX THESE UP

The same word covers two different jobs with different formats, different goals and different consequences.

FOR INFERENCE

- Goal is fewer bytes moved per token
- AWQ, GPTQ, FP8, GGUF k-quants
- The weights are final and frozen
- Quality loss is permanent, so measure it

FOR TRAINING, QLoRA

- Goal is fitting the run on one GPU
- NF4 with double quantization
- The base is frozen, adapters are not
- You can merge and re-export afterwards



A common and reasonable workflow: **train with QLoRA in NF4, merge the adapter into the base at full precision, then quantize the merged model with AWQ for serving.** Different formats for different jobs.



Never serve an NF4 training checkpoint directly. It was chosen for gradient friendliness, not for fast inference kernels.



The Whole Deck In One Table

RECAP

The lines worth keeping when you next have to choose a number of bits.

TOPIC	THE LINE THAT MATTERS
Sizing	params times bytes, then add 20 to 40 percent
Formats	BF16 for range, FP16 for precision, INT4 to fit
Budget	weights are only one of three lines
Why it works	weights cluster near zero, networks are redundant
Why it breaks	a few outlier channels carry the load
Methods	AWQ or FP8 on a GPU, GGUF on a laptop
Q4_K_M	4 bit k-quant, medium variant, the default
Speed	from fewer bytes moved, not from less maths
Cache	quantize it before dropping to 3 bit weights
Calibration	use a few hundred of your own prompts
Group size	128 by default, 64 when quality is tight
Evaluation	task accuracy, never perplexity alone



The one rule that beats all the others: **at a fixed memory budget, a bigger model at 4 bit usually beats a smaller model at 16 bit.**



AI Coach John

FOLLOW

 REPOST